

Fundamentos de Infraestruturas de Computação na Nuvem

Infraestruturas de Computação na Nuvem, Aula 01

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Parte 1

O objeto de estudo

O que distingue uma infraestrutura

- Uma infraestrutura caracteriza-se por ser **partilhada, persistente e transparente em funcionamento normal**.
- Partilhada: serve múltiplos consumidores que não coordenam entre si.
- Persistente: a sua escala de tempo é superior à das aplicações que suporta.
- Transparente: quando funciona bem, ninguém repara nela. Torna-se visível apenas na falha – o que tem consequências para a forma como é financiada e valorizada.

O problema

- Existe um conjunto *finito* de recursos físicos e um conjunto *variável* de procuras concorrentes sobre eles.
- Toda a disciplina decorre da tensão entre estes dois factos.
- As perguntas que atravessam o semestre são sempre variações da mesma: *como se reparte um recurso escasso entre consumidores que não confiam uns nos outros, de forma eficiente, previsível e resistente a falhas?*

Partilhar sem interferir.

Tipos de recurso

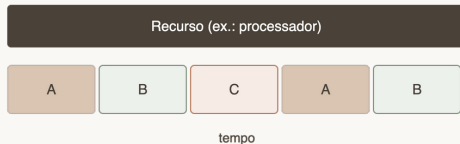
- **Recurso**: entidade finita cujo consumo por um utilizador reduz a disponibilidade para os restantes.
- Distinguem-se pela forma como se esgotam e se recuperam:
 - **Renováveis por tempo** – processador, largura de banda. Não usar num instante é desperdício irrecuperável.
 - **Ocupacionais** – memória, espaço em disco. Mantêm-se ocupados até serem libertados.
- Esta distinção determina que mecanismo de partilha é aplicável a cada recurso.

Dois mecanismos de partilha

- **Multiplexagem temporal:** o recurso é atribuído integralmente a um consumidor de cada vez, alternando no tempo. Aplica-se a recursos renováveis por tempo.
- **Multiplexagem espacial:** o recurso é dividido em porções atribuídas simultaneamente. Aplica-se a recursos ocupacionais.
- A comutação temporal tem um custo de *troca de contexto*; a divisão espacial tem um custo de *fragmentação*.
- Praticamente todos os mecanismos que estudaremos são combinações destes dois, aplicadas a níveis diferentes de abstração.

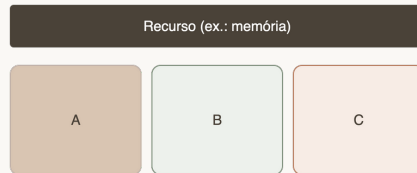
Multiplexagem temporal e espacial

Multiplexagem temporal



O recurso é atribuído por inteiro, um consumidor de cada vez.
Custo: troca de contexto a cada comutação.

Multiplexagem espacial



O recurso é dividido em porções atribuídas em simultâneo.
Custo: fragmentação do espaço não utilizado.

Virtualizar não é só criar VMs

- **Virtualizar** é apresentar a um consumidor uma interface que se comporta como um recurso dedicado, sendo na realidade uma parte de um recurso partilhado.
- O conceito não se restringe a máquinas virtuais: memória virtual, sistemas de ficheiros e endereçamento de rede são todos instâncias do mesmo princípio.
- Requer três capacidades: *traduzir* referências virtuais em físicas, *arbitrar* o acesso concorrente e *isolar* os consumidores entre si.
- Quando qualquer uma das três falha, a abstração deixa de se sustentar – e o comportamento torna-se difícil de explicar a partir da interface.

A virtualização não é uma técnica recente

- Os mainframes IBM System/360, na década de 1960, já particionavam um único computador entre múltiplos sistemas operativos monoutilizador – o antecedente direto da máquina virtual atual.
- O interesse esmoreceu nos anos 1980 e 1990, quando os sistemas operativos multiutilizador tornaram redundante grande parte dessa necessidade.
- Foi retomada a partir de 1998, com a VMware, e consolidada entre 2005 e 2008 com suporte de hardware dedicado (Intel VT-x, AMD-V) e hypervisors como Xen e KVM.
- É esta maturação de várias décadas – não uma inovação recente – que sustenta o modelo de *cloud* elástica discutido na Parte 4.

A definição formal de Popek e Goldberg

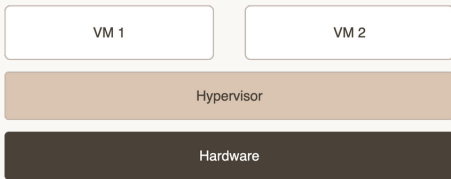
- Referência clássica: Gerald J. Popek e Robert P. Goldberg, *Formal Requirements for Virtualizable Third Generation Architectures*, Communications of the ACM, 1974.
- Os autores formalizam a exigência de isolamento enunciada atrás: um *Virtual Machine Monitor* pode ser construído sempre que o conjunto de instruções sensíveis de uma arquitetura for subconjunto do conjunto de instruções privilegiadas.
- Quando essa condição falha – como sucedia em arquiteturas x86 anteriores a 2005 –, algumas instruções sensíveis não geram exceção, e o isolamento tem de ser assegurado por outros meios.
- O resultado é uma condição verificável sobre uma arquitetura de processador, e não uma propriedade de um produto concreto.

Hypervisor: duas arquiteturas

- **Hypervisor**: o componente que implementa a multiplexagem de hardware entre máquinas virtuais.
- **Tipo 1 (nativo)** – executa diretamente sobre o hardware, sem sistema operativo hospedeiro. Exemplos: Xen, KVM, VMware ESXi.
- **Tipo 2 (alojado)** – executa como aplicação sobre um sistema operativo hospedeiro, que continua a mediar o acesso ao hardware. Exemplo: VirtualBox.
- A diferença é, também ela, uma questão de direcção: o tipo 2 introduz uma camada adicional – o sistema operativo hospedeiro – entre o hypervisor e o hardware.

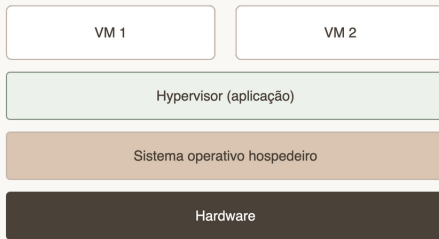
Duas arquiteturas de hypervisor

Tipo 1 — nativo (bare-metal)



O hypervisor acede diretamente ao hardware.
Um nível de indireção; maior desempenho e isolamento.

Tipo 2 — alojado (hosted)



O hypervisor corre como aplicação sobre um sistema operativo hospedeiro: um nível de indireção adicional.

Acréscitar um nível de indirecção

- Introduzir um nível de indirecção significa deixar de referir um recurso diretamente e passar a referi-lo através de um intermediário que faz a tradução.
- É o que permite substituir, mover ou replicar o recurso sem alterar quem o refere.
- Praticamente todas as propriedades que valorizamos – portabilidade, elasticidade, tolerância a falhas – decorrem da existência dessa camada intermédia.
- Formulação clássica, atribuída a David Wheeler: qualquer problema em informática pode ser resolvido acrescentando um nível de indirecção.

O que a indirectão custa

- A mesma formulação costuma ser completada com uma ressalva: o problema que a indirectão não resolve é o de haver indirectão a mais.
- Cada camada acrescenta latência, consome recursos e introduz um novo modo de falha.
- Acrescenta também **opacidade**: o diagnóstico passa a exigir atravessar todas as camadas.
- Decisão de engenharia recorrente nesta UC: *que indirectão compensa introduzir, e a que custo.*
- Não existe resposta universal – depende dos requisitos de cada sistema.

Parte 2

Princípios de arquitetura

Onde as abstrações falham

- Uma abstração é um contrato: expõe um conjunto de operações e oculta a forma como são realizadas.
- O seu valor está no que oculta; a sua fragilidade está exatamente no mesmo sítio.
- **Abstrações com fugas:** propriedades da implementação que se tornam observáveis através da interface – tipicamente o desempenho.
- Um disco virtual comporta-se como um disco até o armazenamento subjacente saturar. A interface não mudou; o comportamento sim.
- Consequência prática: o administrador de infraestrutura tem de compreender *pelo menos uma camada abaixo* daquela em que opera.

Mecanismo e política

- **Mecanismo:** o que o sistema é *capaz* de fazer. **Política:** o que o sistema *deve* fazer em cada circunstância.
- Separá-los permite alterar a política sem reimplementar o sistema.
- Exemplo estrutural: um orquestrador fornece o mecanismo de colocar cargas de trabalho em nós; a política de colocação é declarada à parte, pelo operador.
- É este princípio que torna uma plataforma *configurável* em vez de meramente *programável*.
- Reaparecerá em praticamente todos os blocos do programa.

O argumento end-to-end

- Já vem do contexto do desenho de sistemas distribuídos.
- Uma função só pode ser garantida de forma completa e correta com conhecimento que reside nos *extremos* da comunicação.
- Implementá-la em camadas intermédias é, no limite, redundante – ainda que possa justificar-se por razões de desempenho.
- Consequência para infraestrutura: a camada de infraestrutura não deve tentar garantir propriedades que só a aplicação pode assegurar.
- Delimita o que é razoável exigir de uma plataforma, e o que tem de permanecer responsabilidade de quem a usa.

Declarativo vs imperativo

- **Imperativa:** descreve-se a *sequência de ações* a executar. O resultado depende do estado inicial.
- **Declarativa:** descreve-se o *estado final pretendido*. Cabe ao sistema determinar as ações necessárias.
- A forma declarativa é composicional e verificável: pode comparar-se a especificação com a realidade observada.
- A forma imperativa não o permite – um conjunto de comandos não se compara com um estado.
- Esta distinção é o fundamento conceptual da automatização moderna de infraestrutura.

Idempotência e convergência

- Uma operação é **idempotente** se o efeito de a aplicar n vezes é indistinguível do de a aplicar uma vez.
- Formalmente: $f(f(x)) = f(x)$ para todo o x do domínio.
- A idempotência é o que permite *repetir com segurança* – e repetir com segurança é o que permite recuperar de falhas parciais sem raciocinar sobre o ponto exato em que a execução foi interrompida.
- **Convergência**: propriedade de um sistema que, aplicando repetidamente a mesma especificação, tende para o estado declarado.
- Sem idempotência, a repetição é perigosa; sem repetição, não há convergência.

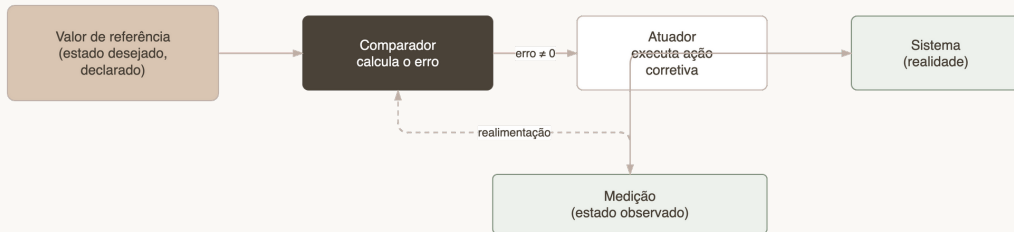
Idempotência na prática

- Suponha um procedimento que acrescenta um servidor a um ficheiro de configuração de um balanceador de carga.
- **Versão não idempotente:** «acrescentar a linha `server 10.0.0.7` ao ficheiro». Correr duas vezes deixa a linha duplicada; o balanceador pode recusar arrancar.
- **Versão idempotente:** «garantir que o ficheiro contém exatamente estes três servidores». Correr as vezes que forem necessárias produz sempre o mesmo ficheiro.
- A diferença aparece quando a execução falha a meio: com a primeira versão é preciso descobrir onde parou; com a segunda, basta repetir.
- É por isso que as ferramentas de automatização que veremos em C3 descrevem *estado*, e não sequências de comandos.

Controlo em malha fechada

- Vários sistemas que estudaremos são, estruturalmente, **controladores em malha fechada**.
- Elementos: um *valor de referência* (estado desejado), uma *medição* (estado observado), um *erro* (a diferença) e uma *ação* que tende a reduzi-lo.
- A execução é contínua, não episódica: o sistema não «faz» e termina – observa e corrige indefinidamente.
- Daqui decorre uma propriedade notável: a recuperação de falhas não precisa de ser programada como caso especial. É o comportamento normal do ciclo.

Como funciona o ciclo



O ciclo é contínuo, não episódico.

A recuperação de falhas não é um caso especial — é o comportamento normal do ciclo.

Acoplamento entre componentes

- **Acoplamento:** medida da dependência entre componentes – quanto é que a alteração ou falha de um afeta os outros.
- Distinguem-se várias dimensões: temporal (têm de estar ativos em simultâneo?), de localização, de formato de dados e de implementação.
- Reduzir acoplamento aumenta a capacidade de evoluir e de falhar isoladamente; aumenta também o número de componentes a gerir.
- Não existe um nível de acoplamento universalmente correto – existe um nível adequado a um dado conjunto de requisitos.

Parte 3

O argumento económico

Porque se consolida

- Um sistema dimensionado para o seu pico de procura permanece subutilizado durante a maior parte do tempo.
- Dimensionar para a média torna-o incapaz de responder ao pico.
- O dilema é insolúvel *para um sistema isolado*. Deixa de o ser quando se agregam vários sistemas independentes.
- A consolidação não é apenas uma conveniência operacional: assenta numa propriedade estatística da agregação de procuras independentes.

Quando a consolidação não compensa

- Agregar muitas procuras independentes torna a procura total proporcionalmente *mais previsível* do que cada uma isoladamente.
- Daí resulta que a margem de segurança necessária, em proporção, diminui à medida que se agregam cargas – é esta a base da economia de escala.
- A hipótese crítica é a **independência**. Se as cargas forem correlacionadas, por exemplo com pico à mesma hora, a previsibilidade desaparece e o ganho com ela.
- Um modelo é útil sobretudo por ser explícito quanto às condições em que deixa de se aplicar.

Utilização e latência

- Uma intuição comum sugere que se deve operar a infraestrutura com utilização próxima de 100%, para não desperdiçar.
- A teoria de filas de espera mostra que essa intuição está errada.
- Para um sistema elementar com chegadas e serviço exponenciais (modelo M/M/1), o tempo médio de resposta R relaciona-se com o tempo de serviço S e a utilização ρ por

$$R = \frac{S}{1 - \rho}$$

- À medida que $\rho \rightarrow 1$, o tempo de resposta cresce sem limite.

O que isto significa na prática

- Avaliando R/S para valores crescentes de utilização:
 - $\rho = 0,50 \Rightarrow R = 2S$
 - $\rho = 0,80 \Rightarrow R = 5S$
 - $\rho = 0,90 \Rightarrow R = 10S$
 - $\rho = 0,99 \Rightarrow R = 100S$
- A degradação não é linear.
- Operar com folga não é desperdício – é o preço da previsibilidade da latência.

O que torna a elasticidade útil

- **Elasticidade:** capacidade de ajustar a capacidade aprovionada à procura observada, em ambos os sentidos.
- O seu valor depende de três grandezas: a amplitude da variação da procura, o tempo de reação do mecanismo de ajuste e a granularidade da unidade faturada.
- Se o tempo de reação for superior à duração do pico, a elasticidade não produz benefício.

Parte 4

O que é, afinal, *cloud*

Porque precisamos de uma definição

- O termo *cloud* é usado com sentidos muito diferentes consoante o contexto – comercial, técnico ou contratual.
- Num contexto de engenharia, um termo sem fronteiras definidas impede raciocínio rigoroso e comparação entre alternativas.
- A definição do NIST (Mell e Grance, 2011) é a referência habitual porque é *operacional*: enuncia critérios verificáveis.
- Vamos analisá-la como se analisa qualquer definição: identificando o que inclui, o que exclui e o que deixa em aberto.

As cinco características do NIST

- **Self-service sob procura** – o consumidor aprovisiona sem intervenção humana do fornecedor.
- **Acesso alargado à rede** – disponível através de mecanismos padronizados.
- **Partilha de recursos** (*resource pooling*) – recursos servem múltiplos consumidores, com atribuição dinâmica.
- **Elasticidade rápida** – capacidade ajustável, em ambos os sentidos, com rapidez.
- **Serviço medido** – o consumo é monitorizado e reportado de forma transparente.

O que a definição realmente diz

- As cinco não são independentes: o *serviço medido* é pré-condição do *self-service* com responsabilização, e a *partilha de recursos* é pré-condição da *elasticidade*.
- Duas são de natureza técnica (*partilha*, *elasticidade*); três são de natureza *organizacional* – descrevem a relação entre fornecedor e consumidor.
- Consequência frequentemente ignorada: uma infraestrutura pode ser tecnicamente sofisticada e ainda assim não constituir uma *cloud*, se o aprovisionamento exigir intervenção humana.
- A definição é, no essencial, sobre **modo de operação**, não sobre tecnologia empregue.

O que significam IaaS, PaaS e SaaS

- **IaaS** — *Infrastructure as a Service*

O fornecedor entrega recursos brutos: capacidade de computação, armazenamento e rede. O cliente instala e gere o sistema operativo e tudo o que corre acima dele.

- **PaaS** — *Platform as a Service*

O fornecedor entrega um ambiente de execução já pronto. O cliente entrega a aplicação e os dados, sem gerir o sistema operativo.

- **SaaS** — *Software as a Service*

O fornecedor entrega a aplicação a funcionar. Ao cliente resta configurar acessos e gerir os seus dados.

IaaS, PaaS e SaaS

- **IaaS, PaaS e SaaS** são habitualmente apresentados como níveis de abstração crescente.
- É mais produtivo lê-los como diferentes **repartições de responsabilidade** sobre uma mesma pilha de camadas.
- Em cada modelo desloca-se a fronteira entre o que o fornecedor gere e o que o consumidor gere.
- Cada deslocação troca *controlo* por *redução de encargo operacional* – e é essa troca, não o nível de abstração em si, que fundamenta a escolha.

Onde fica a fronteira



O que a fronteira determina

- A fronteira determina simultaneamente a responsabilidade *operacional* e a responsabilidade *de segurança*.
- Uma fonte recorrente de incidentes é a suposição de que o que está acima da fronteira é assegurado por quem está abaixo dela.
- A fronteira determina também o **grau de dependência do fornecedor**: quanto mais alto o modelo, mais específicas as interfaces utilizadas, e mais dispendiosa a migração.
- Esta unidade curricular situa-se predominantemente ao nível de **IaaS** – a camada onde a infraestrutura é explicitamente construída e administrada.

Pública, privada, híbrida

- **Pública, privada, híbrida e comunitária** distinguem-se pelo *locus de controlo* e pelo conjunto de consumidores servidos.
- A distinção não é primariamente técnica: as mesmas tecnologias podem sustentar qualquer um dos modelos.
- Os critérios de escolha são tipicamente de natureza regulatória, contratual ou de governação – por exemplo, requisitos de localização dos dados.
- O modelo híbrido introduz uma dificuldade específica: a necessidade de manter coerência operacional entre ambientes com modelos de gestão distintos.

O que a definição deixa de fora

- Nada diz sobre **desempenho**: um sistema lento pode satisfazer integralmente as cinco características.
- Nada diz sobre **dependabilidade**: a definição é ortogonal à fiabilidade do serviço.
- Nada diz sobre **custo**: a elasticidade não implica economia.
- Estas dimensões, ausentes da definição, são precisamente aquelas em que reside o trabalho de engenharia – e o objeto das partes seguintes desta aula.

Parte 5

Qualidade e dependabilidade

Quando um atributo se torna requisito

- Requisitos *funcionais* dizem o que o sistema faz; *atributos de qualidade* dizem com que propriedades o faz.
- Em infraestrutura, são os atributos de qualidade que determinam a arquitetura – a funcionalidade é frequentemente trivial.
- Um atributo só é um requisito quando é **quantificado e verificável**. «O sistema deve ser fiável» não é um requisito: não é possível verificar se foi cumprido.
- Tornar um atributo operacional exige definir a grandeza medida, o método de medição e o valor de referência.

Os atributos de dependabilidade

- Quadro de referência estabelecido por Avižienis, Laprie, Randell e Landwehr (2004).
- **Dependabilidade**: capacidade de um sistema merecer confiança justificada no serviço que presta.
- Decompõe-se em atributos mensuráveis:
 - *Disponibilidade* – prontidão para prestar serviço correto.
 - *Fiabilidade* – continuidade do serviço correto ao longo do tempo.
 - *Integridade* – ausência de alterações impróprias do estado.
 - *Manutenibilidade* – aptidão para ser reparado e modificado.

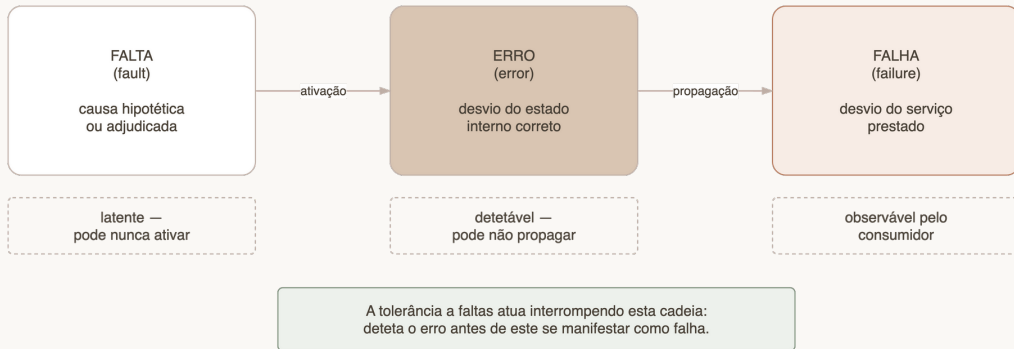
Disponibilidade não é fiabilidade

- **Disponibilidade** é uma proporção de tempo; **fiabilidade** é a probabilidade de funcionamento ininterrupto durante um intervalo.
- Um sistema que falha frequentemente mas recupera em segundos pode ter disponibilidade elevada e fiabilidade baixa.
- Um sistema que falha raramente mas demora dias a recuperar pode ter fiabilidade elevada e disponibilidade baixa.
- A distinção é determinante: os dois atributos exigem investimentos diferentes – respetivamente, em recuperação rápida e em prevenção de falhas.

Falta, erro e falha

- **Falta** (*fault*) – a causa adjudicada ou hipotética de um erro. Pode permanecer latente indefinidamente.
- **Erro** (*error*) – desvio do estado do sistema relativamente ao estado correto. É a manifestação interna da falta.
- **Falha** (*failure*) – o serviço prestado desvia-se do serviço correto. É a manifestação externa do erro.
- A cadeia *não* é determinística: uma falta pode nunca produzir erro, e um erro pode nunca propagar-se até à falha.
- Os mecanismos de tolerância a falhas atuam precisamente interrompendo esta cadeia.

Como uma falta chega a falha



Modelos de falta

- Um sistema tolerante a falhas é-o *sempre relativamente a um modelo de falta declarado*. Não existe tolerância genérica.
- Modelos habituais, por ordem crescente de severidade:
 - **Paragem** (*crash*) – o componente deixa de responder e não volta a agir.
 - **Omissão** – o componente ocasionalmente não responde.
 - **Temporal** – responde, mas fora do intervalo esperado.
 - **Arbitrária** (bizantina) – pode exibir qualquer comportamento, incluindo respostas incorretas e inconsistentes.
- Quanto mais severo o modelo tolerado, mais dispendioso o mecanismo exigido.

Quatro formas de lidar com faltas

- **Prevenção de faltas** – impedir a sua introdução (processos de desenho, revisão, normas).
- **Tolerância a faltas** – assegurar serviço correto na presença de faltas (redundância, detecção e recuperação).
- **Remoção de faltas** – reduzir número e severidade (verificação, teste, diagnóstico).
- **Previsão de faltas** – estimar incidência e consequências (modelos, análise estatística).
- Uma arquitetura séria emprega os quatro. Concentrar o esforço apenas na tolerância é um erro comum e dispendioso.

Quando a redundância não chega

- A tolerância a faltas assenta em redundância – de componentes, de informação ou de tempo.
- A sua eficácia depende de uma hipótese frequentemente implícita: a **independência das faltas**.
- Réplicas que partilham alimentação, rede, versão de software ou defeito de desenho falham em conjunto. A redundância aparente não se traduz em tolerância real.
- **Falta de modo comum**: aquela que afeta simultaneamente todas as réplicas. É o limite fundamental de qualquer esquema de redundância.

SLI, SLO e SLA

- **SLI** (*Service Level Indicator*) – uma grandeza efetivamente medida, com definição operacional precisa.
- **SLO** (*Service Level Objective*) – um valor de referência para esse indicador, assumido internamente.
- **SLA** (*Service Level Agreement*) – um compromisso contratual, com consequências definidas em caso de incumprimento.
- A ordem importa: sem indicador não há objetivo verificável, e sem objetivo o acordo é inexecutável.
- Confundir os três é uma das origens mais comuns de conflito entre equipes técnicas e a organização.

Error budget

- Um SLO inferior a 100% define, por complemento, uma quantidade admissível de indisponibilidade: o **orçamento de erro**.
- Um objetivo de 99,9% num trimestre corresponde a um orçamento de aproximadamente 2,2 horas nesse período.
- A sua função é converter uma discussão qualitativa – «devemos arriscar esta alteração?» – numa decisão com critério explícito.
- Enquanto houver orçamento, a mudança é admissível; esgotado o orçamento, a prioridade desloca-se para a estabilização.
- Torna visível que fiabilidade e velocidade de mudança são grandezas em tensão, e não objetivos independentes.

Atributos que competem entre si

- Atributos de qualidade raramente são independentes; melhorar um degrada tipicamente outro.
 - Consistência forte entre réplicas custa latência.
 - Isolamento forte entre consumidores custa eficiência de utilização.
 - Redundância custa recursos e complexidade operacional.
 - Segurança custa, quase sempre, conveniência.
- Não há arquitetura ótima em abstrato – há arquiteturas adequadas a uma ponderação declarada de requisitos.
- **Explicitar a ponderação é parte do trabalho de engenharia**, não um preliminar dele.

Parte 6

O fio condutor do semestre

Como os blocos se encadeiam

- **C1** estabelece o substrato: recursos finitos, restrições físicas, modelos de operação.
- **C2** introduz o mecanismo que permite parti-los: virtualização e contentorização.
- **C3** trata da gestão em escala: orquestração, rede programável, automatização.
- **C4** passa dos recursos aos serviços: disponibilidade, escala, desacoplamento.
- **C5** trata do estado, que é o que resiste à abstração: armazenamento.
- **C6** fecha o ciclo: observar e administrar o que foi construído.

Perguntas que vão reaparecer

- Que recurso está a ser partilhado, e por que mecanismo?
- Que abstração está a ser oferecida, e em que condições deixa de se sustentar?
- Que modelo de falta é tolerado, e a que custo?
- O estado desejado é declarado ou construído por uma sequência de ações?
- Que indicador permite verificar que o sistema está a fazer o que se pretende?
- Reconhecer estas questões em contextos novos é, mais do que memorizar tecnologias, o objetivo da UC.